- `scaffold_controller` different from `resource_controller`, defines which generator to use for generating a *scaffolded* controller when using `rails generate scaffold`. Defaults to `:scaffold_controller`.
- `stylesheets` turns on the hook for stylesheets in generators. Used in Rails for when the `scaffold` generator is run, but this hook can be used in other generates as well. Defaults to `true`.
- `stylesheet_engine` configures the stylesheet engine (for eg. sass) to be used when generating assets. Defaults to `:css`.
- `test_framework` defines which test framework to use. Defaults to `false` and will use Test::Unit by default.
- `template_engine` defines which template engine to use, such as ERB or Haml. Defaults to `:erb`.

3.4 Configuring Middleware

Every Rails application comes with a standard set of middleware which it uses in this order in the development environment:

- `ActionDispatch::SSL` forces every request to be under HTTPS protocol. Will be available if `config.force_ssl` is set to `true`. Options passed to this can be configured by using `config.ssl_options`.
- `ActionDispatch::Static` is used to serve static assets. Disabled if `config.serve_static_files` is `false`.
- `Rack::Lock` wraps the app in mutex so it can only be called by a single thread at a time. Only enabled when `config.cache_classes` is `false`.
- `ActiveSupport::Cache::Strategy::LocalCache` serves as a basic memory backed cache. This cache is not thread safe and is intended only for serving as a temporary memory cache for a single thread.
- `Rack::Runtime` sets an `X-Runtime` header, containing the time (in seconds) taken to execute the request.
- `Rails::Rack::Logger` notifies the logs that the request has begun. After request is complete, flushes all the logs.
- `ActionDispatch::ShowExceptions` rescues any exception returned by the application and renders nice exception pages if the request is local or if `config.consider_all_requests_local` is set to `true`. If `config.action_dispatch.show_exceptions` is set to `false`, exceptions will be raised regardless.
- `ActionDispatch::RequestId` makes a unique X-Request-Id header available to the response and enables the `ActionDispatch::Request#uuid` method.
- `ActionDispatch::RemoteIp` checks for IP spoofing attacks and gets valid `client_ip` from request headers. Configurable with the `config.action_dispatch.ip_spoofing_check`, and `config.action_dispatch.trusted_proxies` options.
- `Rack::Sendfile` intercepts responses whose body is being served from a file and replaces it with a server specific X-Sendfile header. Configurable with `config.action_dispatch.x_sendfile_header`.
- `ActionDispatch::Callbacks` runs the prepare callbacks before serving the request.
- `ActiveRecord::ConnectionAdapters::ConnectionManagement` cleans active connections after each request, unless the `rack.test` key in the request environment is set to `true`.